Part 1: Java Streams Fundamentals

This is one of the **most important Java 8 topics** for interviews. Streams are used extensively in **Spring Boot**, **microservices**, and **enterprise applications** for processing collections in a clean, declarative way.

---

1. Understand Stream Source, Intermediate Operations, and Terminal Operations

What is a Stream?

A **Stream** is a sequence of elements that supports operations (like filtering, mapping, sorting, etc.) to process data.

> **Important:** A Stream **does not store data**. It only processes data from a source.

Think of it like a **water pipe**:

```
Collection (Source)
        |
        ▼
Intermediate Operations
(filter, map, sorted...)
        |
        ▼
Terminal Operation
(forEach, collect, count...)
        |
        ▼
Final Result
```

Stream Source

A stream always starts from a **source**.

Common sources:

- List
- Set
- Map (keys, values, entries)
- Arrays
- Files
- Generated streams

Example 1: List Source

```
List names = List.of("Ali", "Ahmed", "Sara");

Stream stream = names.stream();
```

Here,

- names → Source
- stream() → Creates a Stream

Example 2: Array Source

```
String[] fruits = {"Apple", "Banana", "Mango"};

Stream stream = Arrays.stream(fruits);
```

Example 3: Stream.of()

```
Stream stream = Stream.of(10, 20, 30);
```

Intermediate Operations

Intermediate operations **transform the stream**.

Examples:

- filter()
- map()
- sorted()
- distinct()
- skip()
- limit()

Characteristics

✅ Return another Stream

✅ Lazy (don't execute immediately)

Example

```
List names = List.of("Ali", "Ahmed", "Sara");

names.stream()
     .filter(name -> name.startsWith("A"));
```

Nothing happens yet!

Terminal Operations

Terminal operations **finish the pipeline** and produce a result.

Examples:

- collect()
- forEach()
- count()
- reduce()
- toArray()
- anyMatch()

Once a terminal operation runs,

the stream is consumed and **cannot be reused**.

Example

```
names.stream()
     .filter(name -> name.startsWith("A"))
     .forEach(System.out::println);
```

Output

```
Ali
Ahmed
```

Complete Pipeline

```
List numbers = List.of(2, 5, 7, 8, 10);

numbers.stream()             // Source
       .filter(n -> n % 2 == 0) // Intermediate
       .map(n -> n * 2)          // Intermediate
       .forEach(System.out::println); // Terminal
```

Output

```
4
16
20
```

Stream Lifecycle

```
Collection
     |
stream()
     |
Intermediate
```

```
     |
Intermediate
     |
Intermediate
     |
Terminal
     |
Finished
```

Real Project Example

Suppose you have employees.

```
employees.stream()
         .filter(Employee::isActive)
         .sorted(Comparator.comparing(Employee::getSalary))
         .map(Employee::getName)
         .forEach(System.out::println);
```

This is much cleaner than nested loops.

Common Beginner Mistakes

Mistake 1

```
Stream s = numbers.stream();
```

Thinking the stream already processed data.

❌ Wrong

Nothing has happened yet.

Mistake 2

```
Stream s = numbers.stream();

s.forEach(System.out::println);

s.forEach(System.out::println);
```

Runtime Exception

```
IllegalStateException
stream has already been operated upon or closed
```

A stream can only be used **once**.

Interview Questions

Q1. Does a Stream store data?

Answer

No.

It only processes data from a source.

Q2. Can a Stream be reused?

No.

Once a terminal operation executes, the stream is closed.

Q3. What are the three parts of a Stream pipeline?

Source → Intermediate → Terminal

One-liner

> A Stream processes data from a source through intermediate operations and finishes with a terminal operation.

---

2. Use filter() for Conditional Selection

filter() is used to **keep only those elements that satisfy a condition**.

It internally uses a **Predicate**.

Signature

```
Stream filter(Predicate predicate)
```

Returns another Stream.

Example 1: Even Numbers

```
List nums = List.of(2,3,4,5,6);

nums.stream()
    .filter(n -> n % 2 == 0)
    .forEach(System.out::println);
```

Output

```
2
4
6
```

Example 2: Adults

```
List ages = List.of(15,18,21,13,30);
```

```
ages.stream()
    .filter(age -> age >= 18)
    .forEach(System.out::println);
```

Output

```
18
21
30
```

Example 3: Names Starting with A

```
List names = List.of("Ali","Ahmed","Sara","Ayesha");

names.stream()
     .filter(name -> name.startsWith("A"))
     .forEach(System.out::println);
```

Output

```
Ali
Ahmed
Ayesha
```

Example 4: Filter Objects

```
class Student {
    String name;
    int marks;

    Student(String name, int marks) {
        this.name = name;
        this.marks = marks;
    }
```

```
}

List students = List.of(
    new Student("Ali", 85),
    new Student("Ahmed", 40),
    new Student("Sara", 92)
);

students.stream()
        .filter(s -> s.marks >= 50)
        .forEach(s -> System.out.println(s.name));
```

Output

```
Ali
Sara
```

Real Project Example

```
users.stream()
     .filter(User::isVerified)
     .forEach(System.out::println);
```

Common Mistake

Wrong

```
numbers.stream()
       .filter(n -> {
            System.out.println(n);
       });
```

filter() **must return a boolean**.

Correct

```
numbers.stream()
       .filter(n -> n > 10);
```

Interview Questions

Why does filter use Predicate?

Because Predicate always returns true or false.

Does filter modify the original List?

No.

Streams never modify the source collection.

Can we chain multiple filters?

Yes.

```
numbers.stream()
       .filter(n -> n > 5)
       .filter(n -> n % 2 == 0);
```

One-liner

> filter() keeps only those elements that satisfy a condition.

---

3. Use map() for Object Transformation

map() converts each element into another value.

It internally uses a **Function**.

Signature

```
 Stream map(Function mapper)
```

Think:

```
Input → Output
```

Example 1: Square Numbers

```
List nums = List.of(2,3,4);

nums.stream()
    .map(n -> n * n)
    .forEach(System.out::println);
```

Output

```
4
9
16
```

Example 2: Uppercase

```
List names = List.of("java","spring");

names.stream()
     .map(String::toUpperCase)
     .forEach(System.out::println);
```

Output

```
JAVA
SPRING
```

Example 3: String Length

```
List words = List.of("Java","Spring","Boot");

words.stream()
     .map(String::length)
     .forEach(System.out::println);
```

Output

```
4
6
4
```

Example 4: Employee → Name

```
employees.stream()
         .map(Employee::getName)
         .forEach(System.out::println);
```

Converts

```
Employee
```

↓

```
String
```

Example 5: Celsius → Fahrenheit

```
List celsius = List.of(0,20,30);

celsius.stream()
       .map(c -> (c * 9/5) + 32)
       .forEach(System.out::println);
```

Output

```
32
68
86
```

Real Project Example

DTO Conversion

```
users.stream()
     .map(UserDTO::new)
     .toList();
```

Very common in Spring Boot.

Common Mistake

Many beginners think

```
map()
```

filters data.

It doesn't.

It **transforms** data.

Interview Questions

Difference between filter and map?

Filter removes elements.

Map transforms elements.

Can map change object type?

Yes.

Example

```
Employee
```

↓

```
String
```

One-liner

> map() transforms each element into another value.

---

4. Use sorted() with Comparator

sorted() sorts stream elements.

Two versions

```
sorted()
```

Natural order

```
sorted(Comparator)
```

Custom order

Example 1: Natural Order

```
List nums = List.of(8,2,6,1);

nums.stream()
    .sorted()
    .forEach(System.out::println);
```

Output

```
1
2
6
8
```

Example 2: Reverse Order

```
nums.stream()
    .sorted(Comparator.reverseOrder())
```

```
    .forEach(System.out::println);
```

Output

```
8
6
2
1
```

Example 3: Sort by Length

```
List words = List.of("Java","C","Spring");

words.stream()
     .sorted(Comparator.comparing(String::length))
     .forEach(System.out::println);
```

Output

```
C
Java
Spring
```

Example 4: Sort Employees by Salary

```
employees.stream()
         .sorted(Comparator.comparing(Employee::getSalary))
         .forEach(System.out::println);
```

Example 5: Descending Salary

```
employees.stream()
         .sorted(
             Comparator.comparing(Employee::getSalary)
                       .reversed()
         )
         .forEach(System.out::println);
```

Real Project Example

```
products.stream()
        .sorted(Comparator.comparing(Product::getPrice))
        .toList();
```

Very common in REST APIs.

Common Mistake

Wrong

```
.sorted()
```

on a custom object that **doesn't implement Comparable**.

This throws a ClassCastException.

Use a Comparator instead.

```
.sorted(Comparator.comparing(Employee::getSalary))
```

Interview Questions

Difference between Comparable and Comparator?

Comparable	Comparator
Inside the class	Outside the class
One default sorting	Multiple custom sortings
compareTo()	compare()

Which sorted() overload should you use for custom objects?

Use

```
sorted(Comparator.comparing(...))
```

Does sorted modify the original List?

No.

It returns a **new sorted stream**.

Interview Summary (Part 1)

Stream Pipeline

```
Source
 ↓
Intermediate
 ↓
Intermediate
 ↓
Terminal
```

Functional Interfaces Used

Stream Operation	Functional Interface

filter()	Predicate
map()	Function
sorted()	Comparator
forEach()	Consumer

Overall Interview Questions (With Short Answers)

Q1. What is a Stream?

Answer: A pipeline for processing data from a source without storing it.

Q2. Does a Stream modify the original collection?

Answer: No.

Q3. Can a Stream be reused?

Answer: No, a stream is single-use.

Q4. What does filter() return?

Answer: A new Stream containing elements that satisfy the condition.

Q5. Which functional interface does filter() use?

Answer: Predicate.

Q6. Which functional interface does map() use?

Answer: Function.

Q7. What is the difference between filter() and map()?

Answer: filter() removes elements; map() transforms elements.

Q8. Which sorted() should be used for custom objects?

Answer: sorted(Comparator.comparing(...)).

Q9. Can map() change the data type?

Answer: Yes (e.g., Employee → String).

Q10. Does sorted() change the original list?

Answer: No.

One-Line Interview Revision

- **Stream:** Processes data from a source through a pipeline.
- **Source:** Where the stream gets its data (List, Set, Array, etc.).
- **Intermediate Operation:** Returns another stream and is lazy (filter, map, sorted).
- **Terminal Operation:** Triggers execution and produces a result (collect, forEach, count).
- **filter():** Selects elements based on a condition.
- **map():** Transforms one value into another.
- **sorted():** Sorts elements using natural order or a Comparator.
- **Stream Rule:** A stream can be consumed only once.